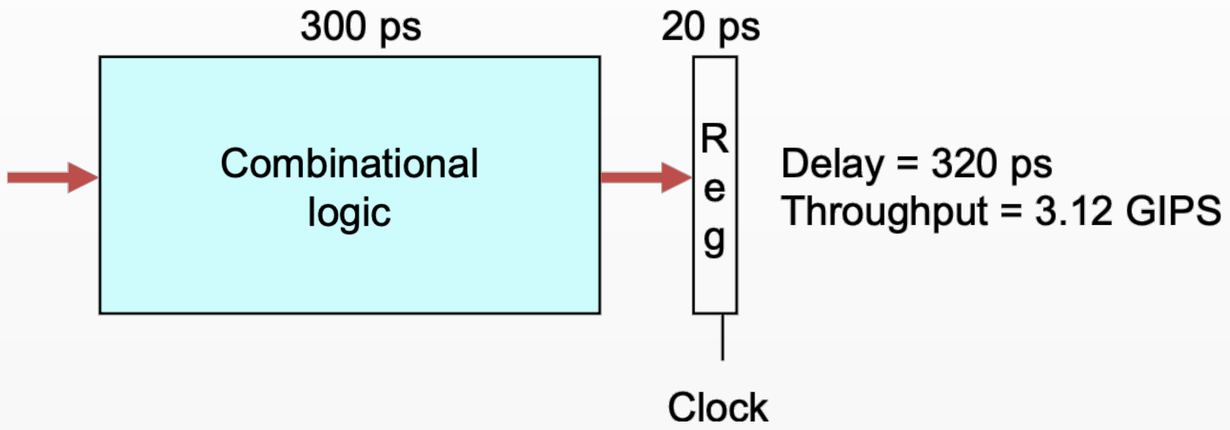
Processor Architecture: Pipelined (I)

王愉博 20251029

1. 性能对比

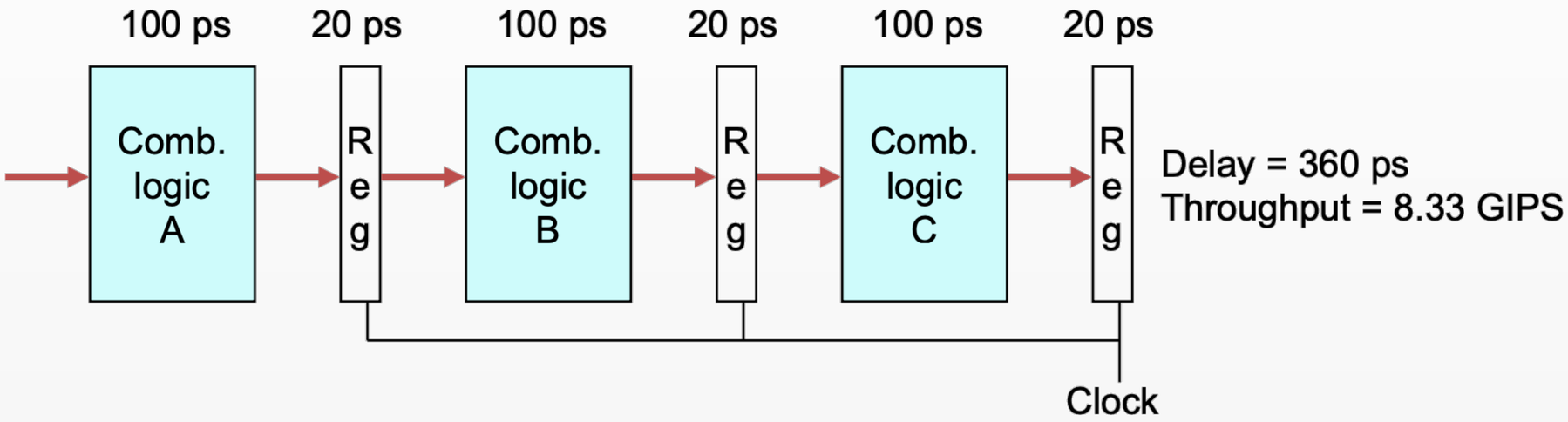
无流水线



- Delay = 300ps + 20ps = 320 ps
- Clock cycle time = 320 ps
- Throughput = 1e12ps / 320ps/Ins = 3.12GI/s (GIPS)

1. 性能对比

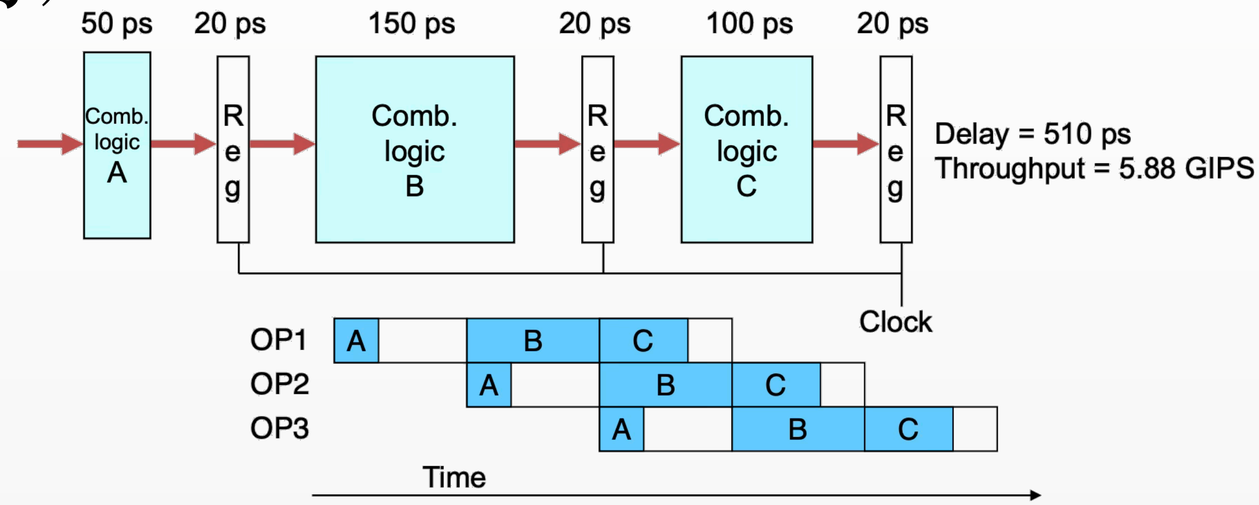
3级流水（均匀）



- Delay = 300ps + 3*20ps = 360 ps
- Clock cycle time = max (120, 120, 120) = 120 ps
- Throughput = 1/Clock cycle time = 8.33 GIPS

1. 性能对比

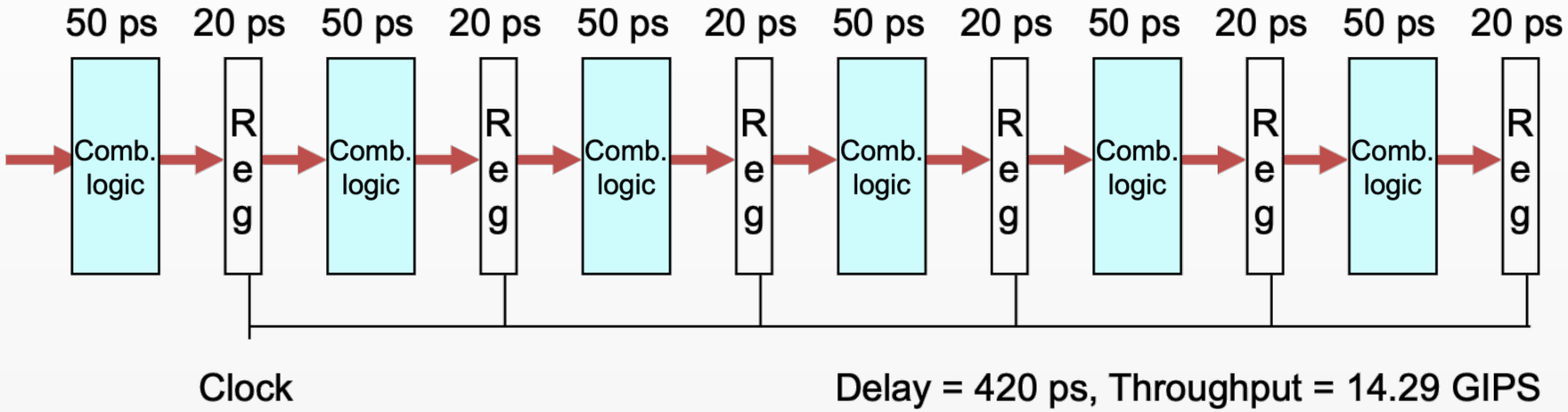
3级流水（不均匀）



- Delay = 50 + 20 +150 + 20 + 100 + 20 = 510 ps
- Clock cycle time = max (70, 170, 120) = 170 ps
- Throughput = 1/Clock cycle time = 5.88 GIPS
- 时钟周期必须能让所有阶段执行完毕，从而由最慢的决定

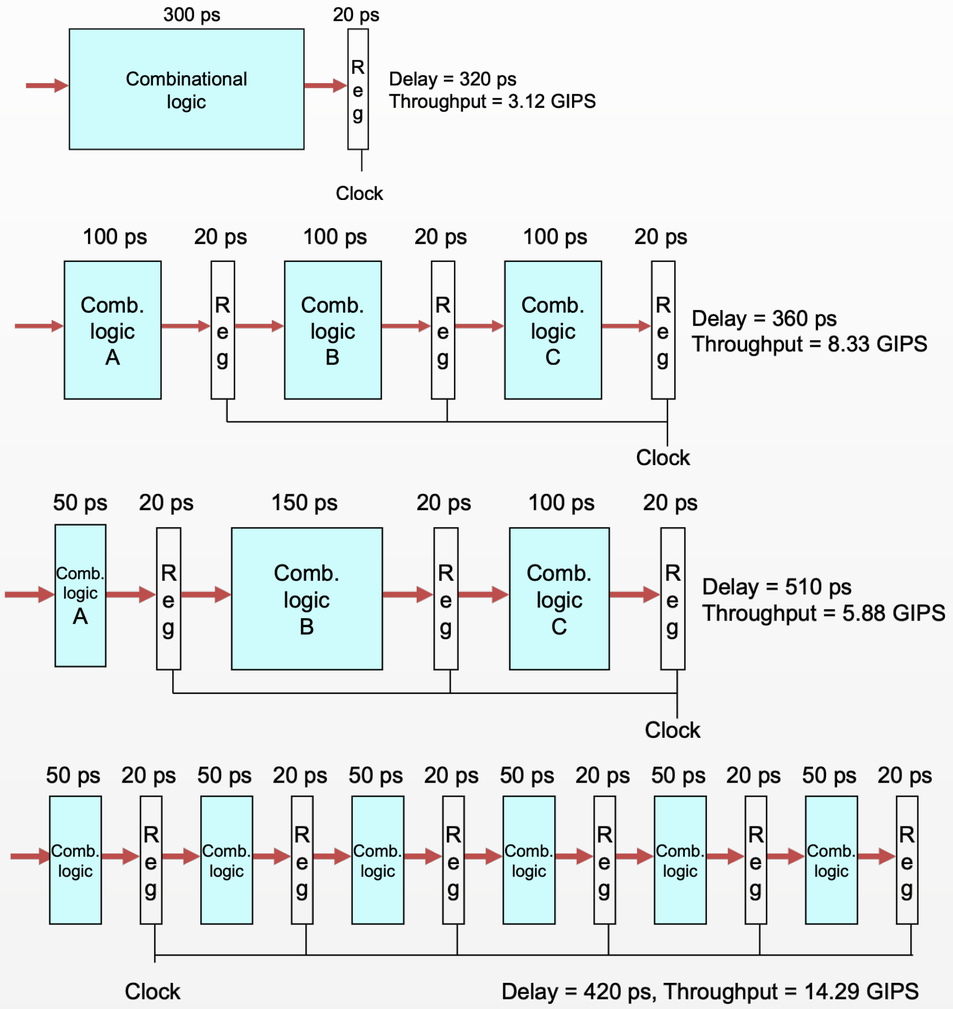
1. 性能对比

6级流水



- Delay = 420 ps
- Clock cycle time = 70 ps
- Throughput = 1/Clock cycle time = 14.29 GIPS

1. 性能对比

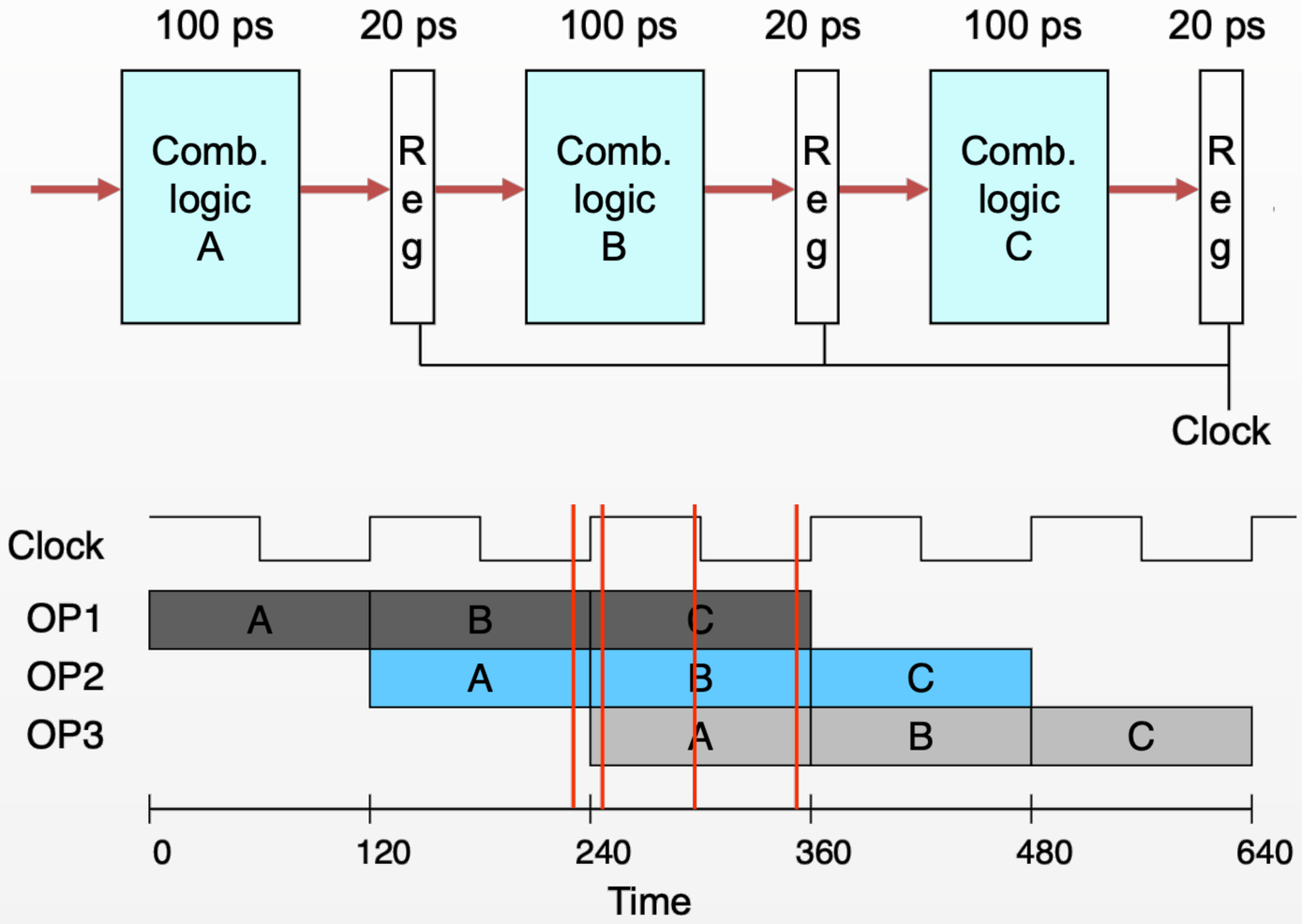


Delay (ps)	Clock cycle (ps)	Throughput (GIPS)
320	320	3.12
360	120	8.33
510	170	14.29
420	70	5.88

1. 性能对比

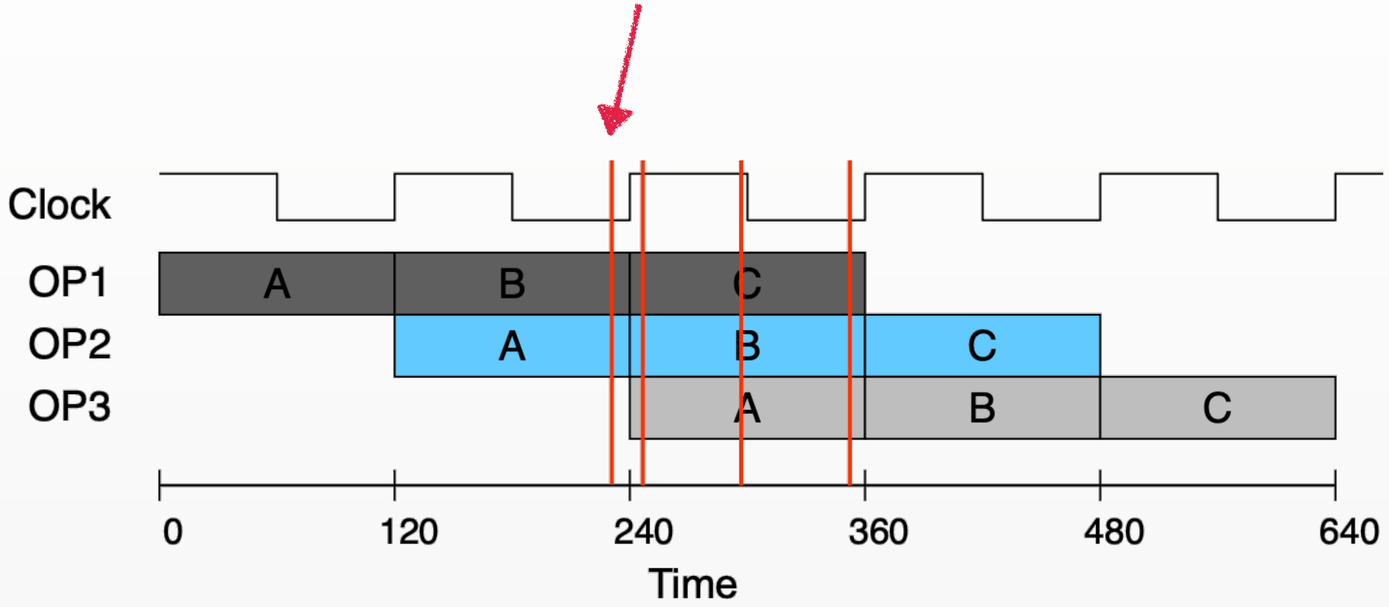
- 增加寄存器个数会增加Delay
- Throughput由Clock cycle time决定
- Clock cycle time由所有阶段耗时的最大值决定
- 通过尽量“平均”地划分组合逻辑阶段，可以最小化Clock cycle time，从而最大化Throughput

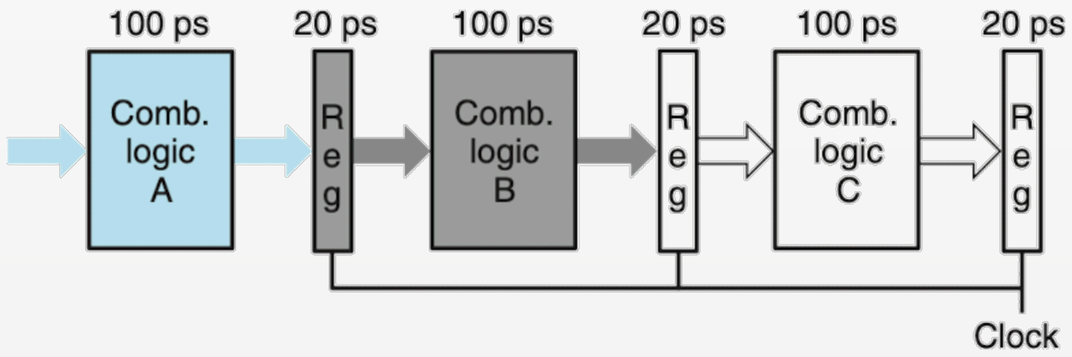
2. 流水线的执行


100 ps
20 ps
100 ps
20 ps
100 ps
20 ps
Comb. logic A
Reg
Comb. logic B
Reg
Comb. logic C
Reg
Clock
Clock
OP1
OP2
OP3
A
B
C
A
B
C
A
B
C
0
120
240
360
480
640
Time

2. 流水线的执行

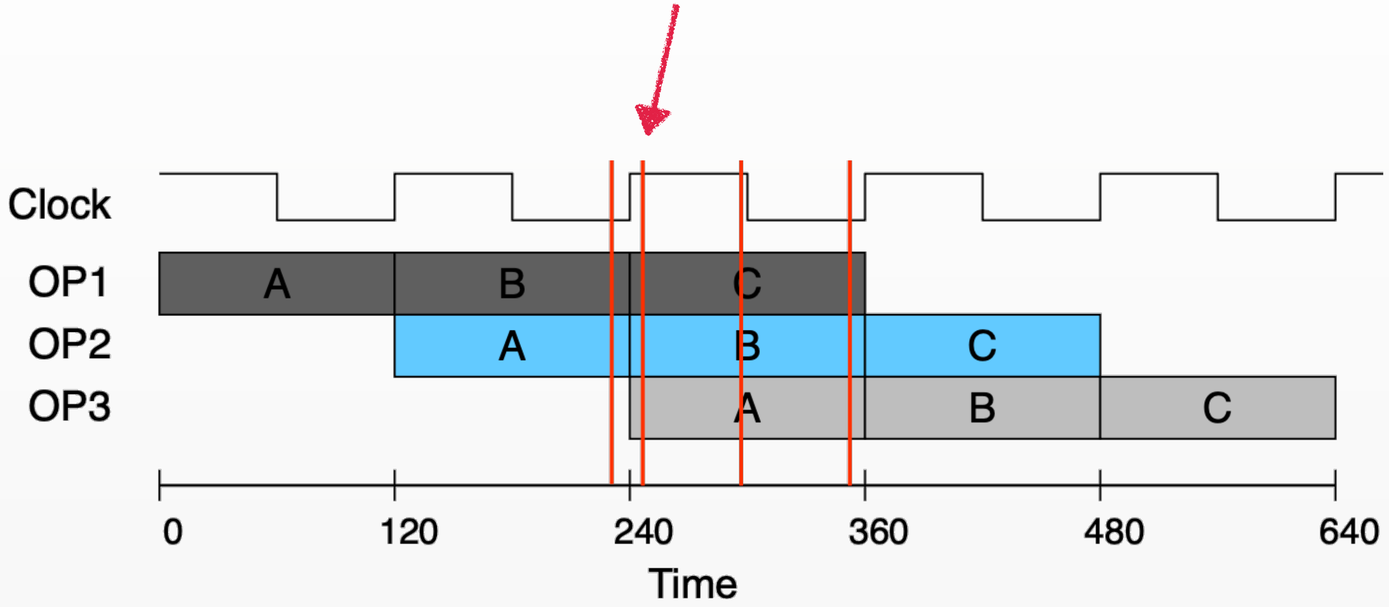
Time = 239

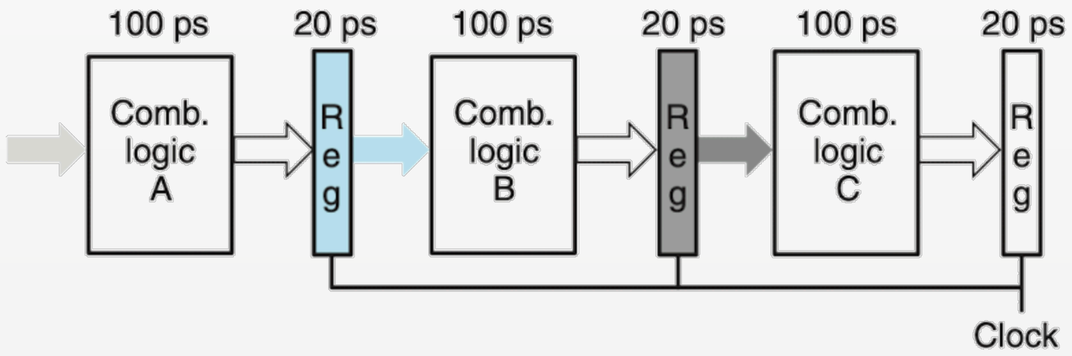




2. 流水线的执行

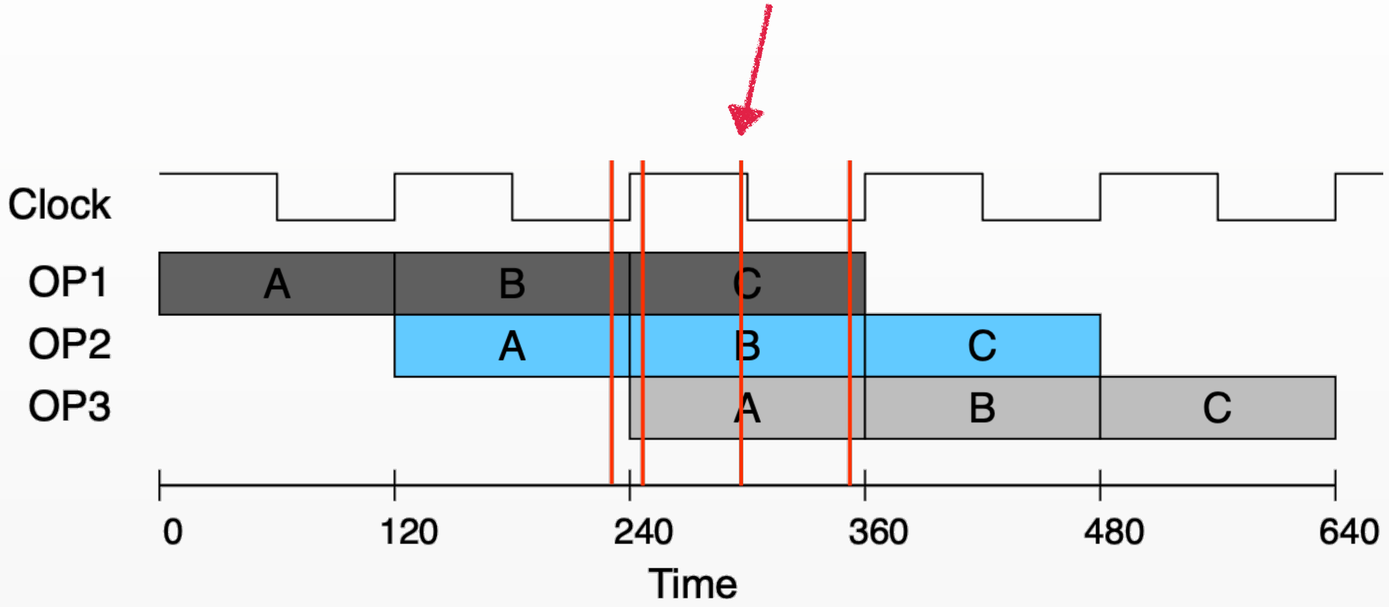
Time = 241

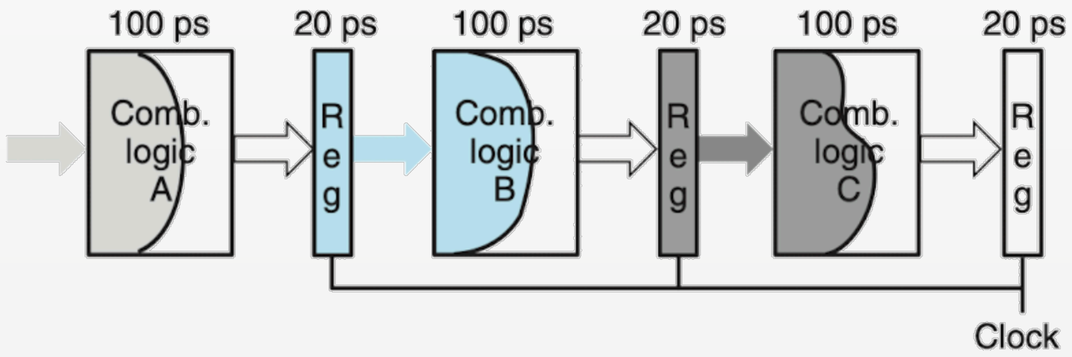




2. 流水线的执行

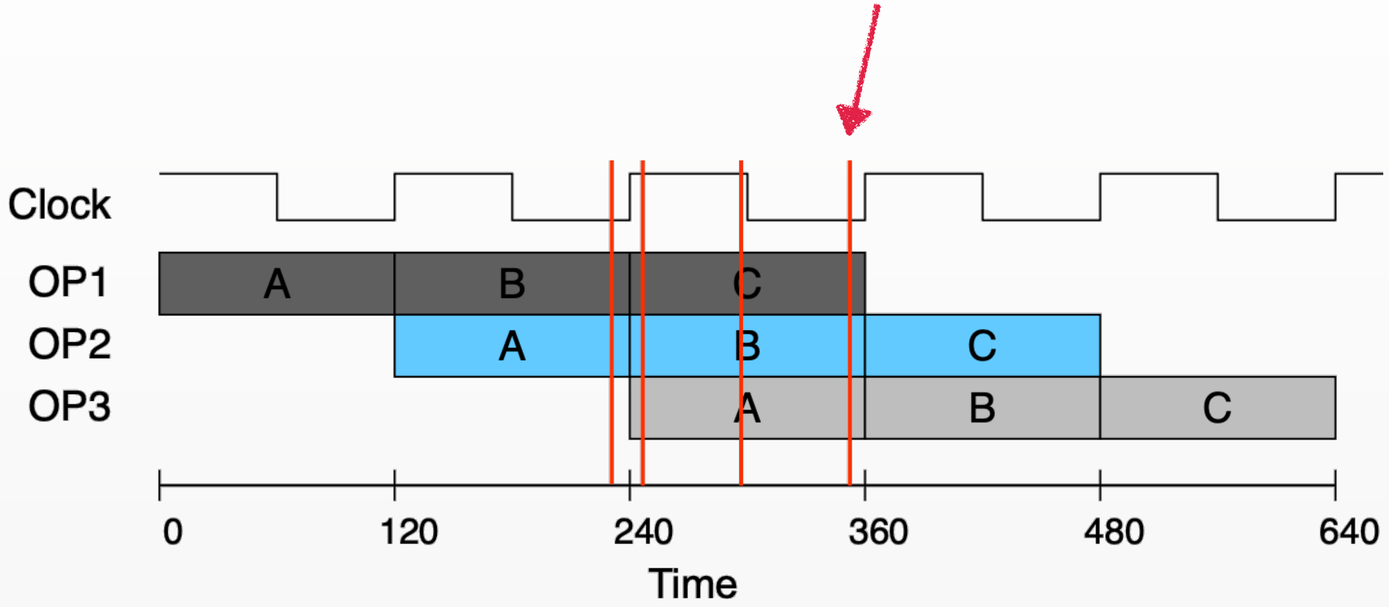
Time = 300

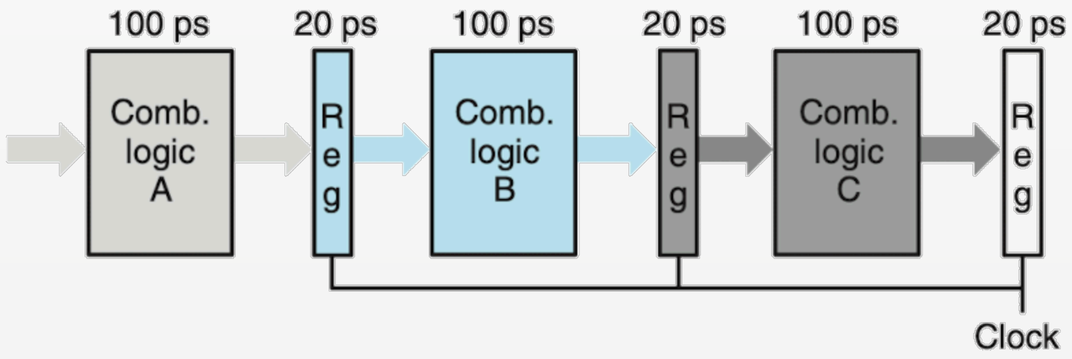




2. 流水线的执行

Time = 359





3. 数据相关 vs 数据冒险

数据相关（Data dependency)

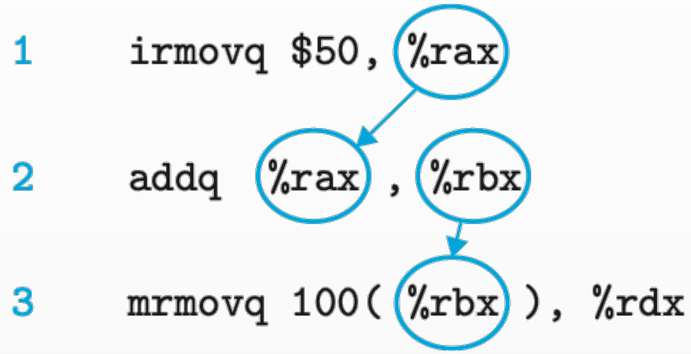
- where the results computed by one instruction are used as the data for a following instruction
- 尝试访问没有就绪的数据
- 程序的静态属性，由指令序列的逻辑决定

3. 数据相关 vs 数据冒险

数据冒险（Data hazard）

- An erroneous computation by the pipeline
- 因数据相关，导致后一条指令在流水线中过早地试图读取或写入数据，从而获取到错误（过时）的值
- 流水线硬件的动态问题，是数据相关在特定流水线执行时序下的产物
- 数据相关是原因，数据冒险是后果

4. RAW Dependency



- addq 的源数据依赖 irmovq 的执行
- mrmovq 源数据依赖 addq 的执行

4. RAW Dependency

RAW导致Data Hazard的情况

- A data hazard can arise for an instruction when one of its operands is updated by any of the **three preceding instructions**.
- 指令在Decode阶段读取数据，前1/2/3条指令分别在E/M/W阶段，分别需要等待3/2/1个cycle才能确保写回完成

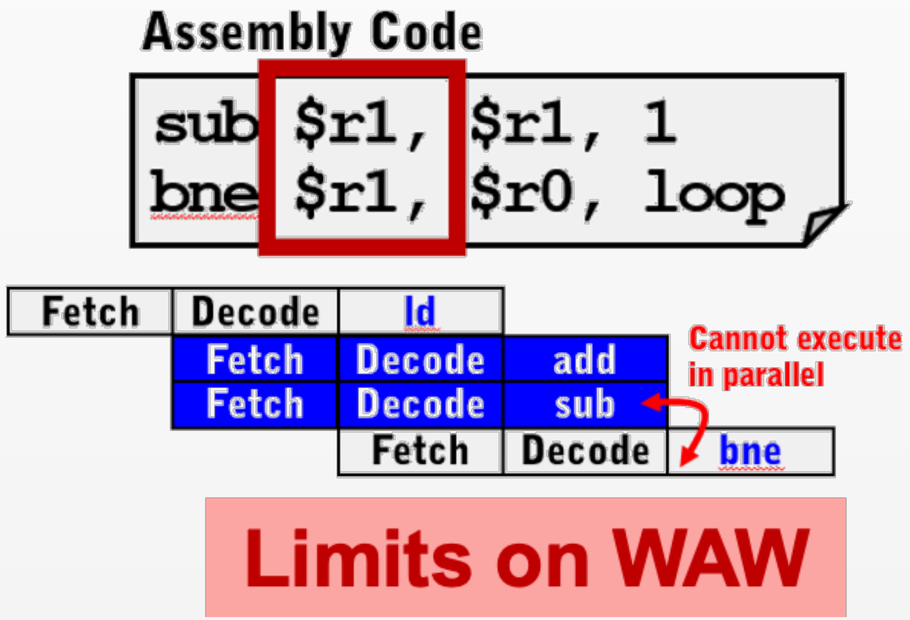
5. Other Data Dependencies

- RAW - 由指令执行时序决定，可能在5级流水线中导致hazard
- WAW - 在标准5级流水线中不会造成hazard
- WAR - 在标准5级流水线中不会造成hazard
- RAR在任何情况下都不会造成hazard

5. Other Data Dependencies

WAW dependency

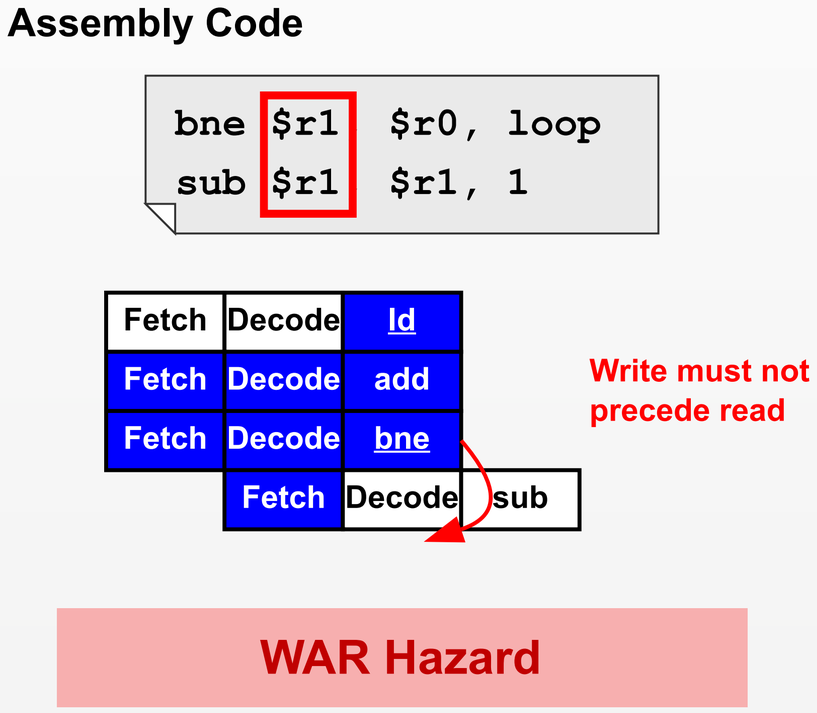
- 考虑multi-issue处理器的情况



5. Other Data Dependencies

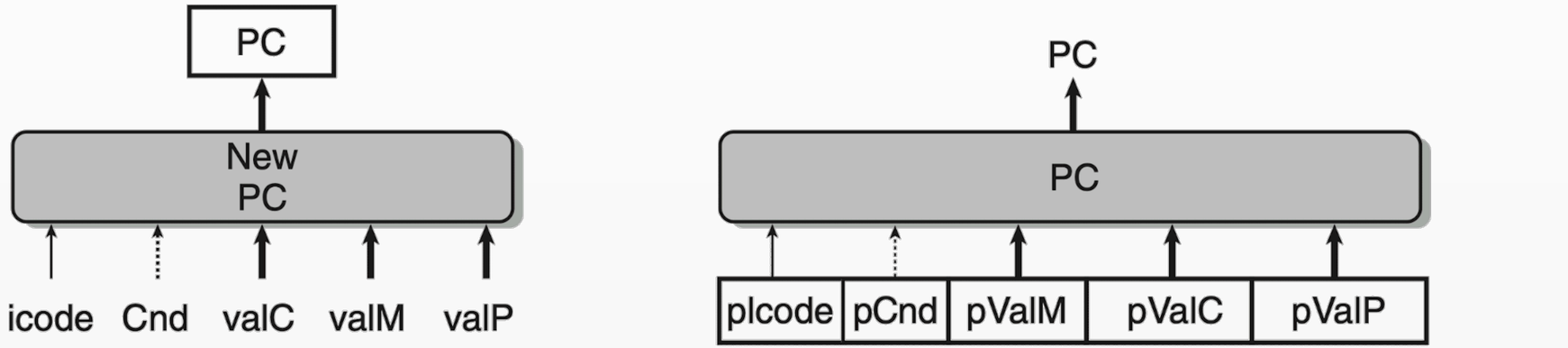
WAR dependency

- 考虑multi-issue处理器的情况



6. SEQ vs SEQ+

将PC预测提前



(a) SEQ new PC computation

(b) SEQ+ PC selection

Figure 4.39 Shifting the timing of the PC computation. With SEQ+, we compute the value of the program counter for the current state as the first step in instruction execution.

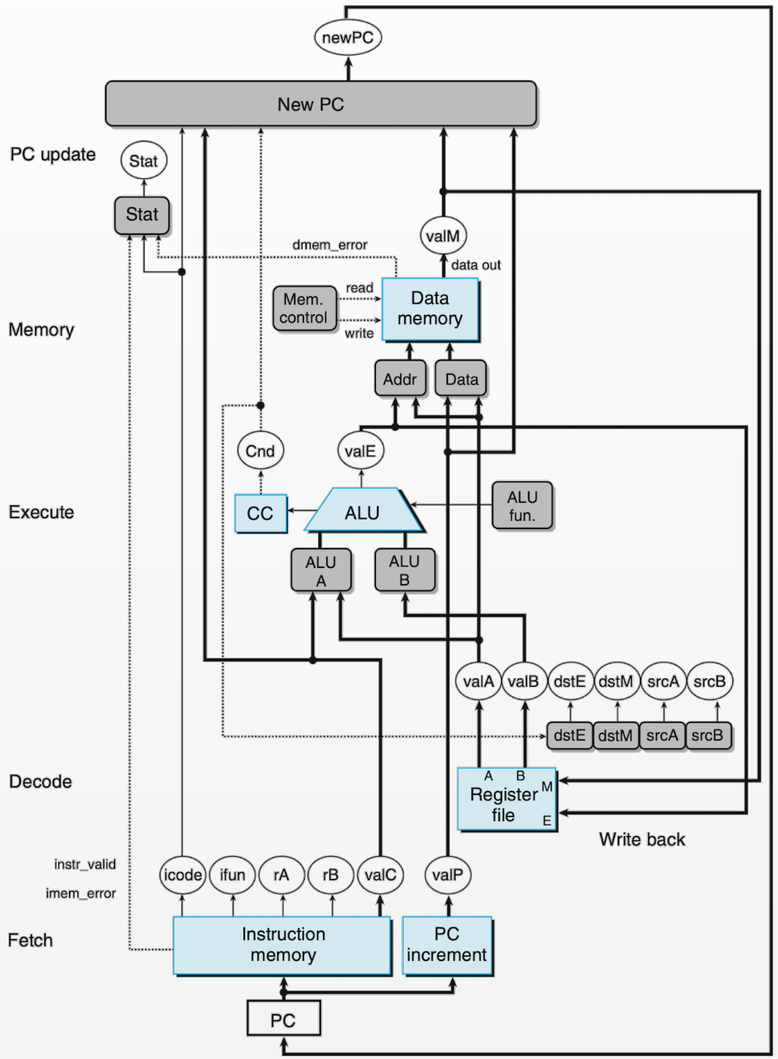
- 取消了储存PC的硬件寄存器，而是动态计算PC
- PC的计算不再依赖之前的结果

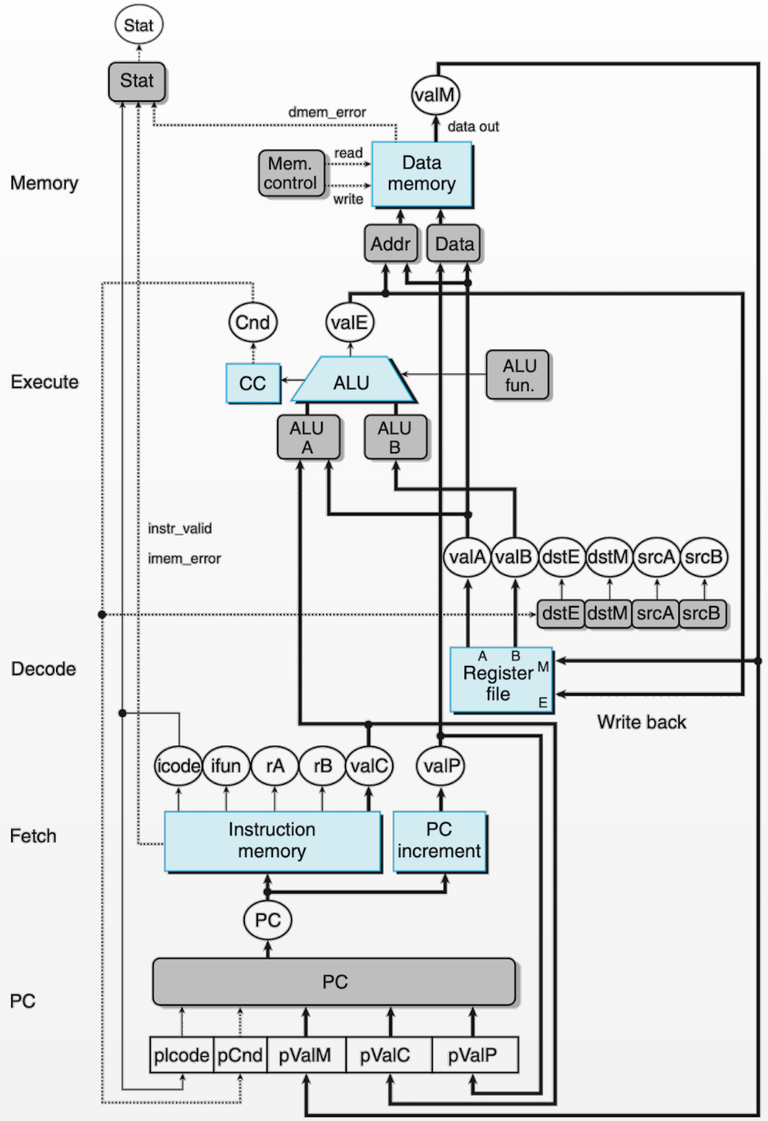
6. SEQ vs SEQ+

将PC预测提前

- Pipeline后续阶段的指令执行都依赖预测的PC
- SEQ中PC预测必须在之前阶段执行之后
- SEQ+中PC预测只依赖硬件寄存器
- 因此改为SEQ+才可以“分出”PC预测的阶段
- 假设在SEQ中插入流水线寄存器

6. SEQ vs SEQ+


newPC
New PC
PC update
Stat
Stat
dmem_error
valM
data out
Memory
Mem. control
read
write
Data memory
Addr
Data
Cnd
valE
Execute
CC
ALU
ALU fun.
ALU A
ALU B
valA
valB
dstE
dstM
srcA
srcB
dstE
dstM
srcA
srcB
Decode
A
B
Register file
M
E
Write back
instr_valid
imem_error
icode
ifun
rA
rB
valC
valP
Fetch
Instruction memory
PC increment
PC


Stat
Stat
valM
dmem_error
data out
Memory
Mem. control
read
write
Data memory
Addr
Data
Cnd
valE
Execute
CC
ALU
ALU fun.
ALU A
ALU B
instr_valid
imem_error
valA
valB
dstE
dstM
srcA
srcB
dstE
dstM
srcA
srcB
Decode
A
B
Register file
M
E
Write back
icode
ifun
rA
rB
valC
valP
Fetch
Instruction memory
PC increment
PC
PC
PC
pIcode
pCnd
pValM
pValC
pValP

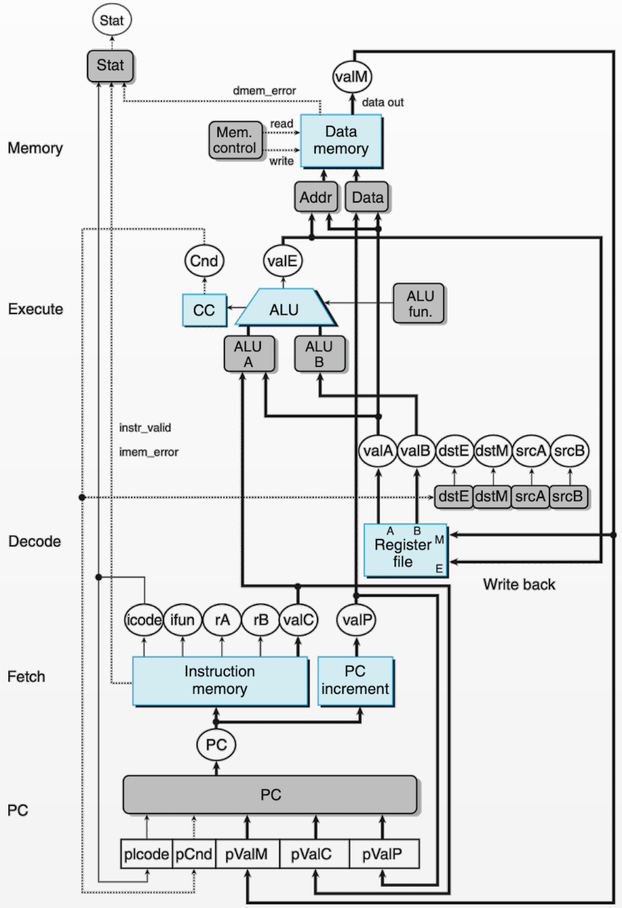
7. 实现流水线

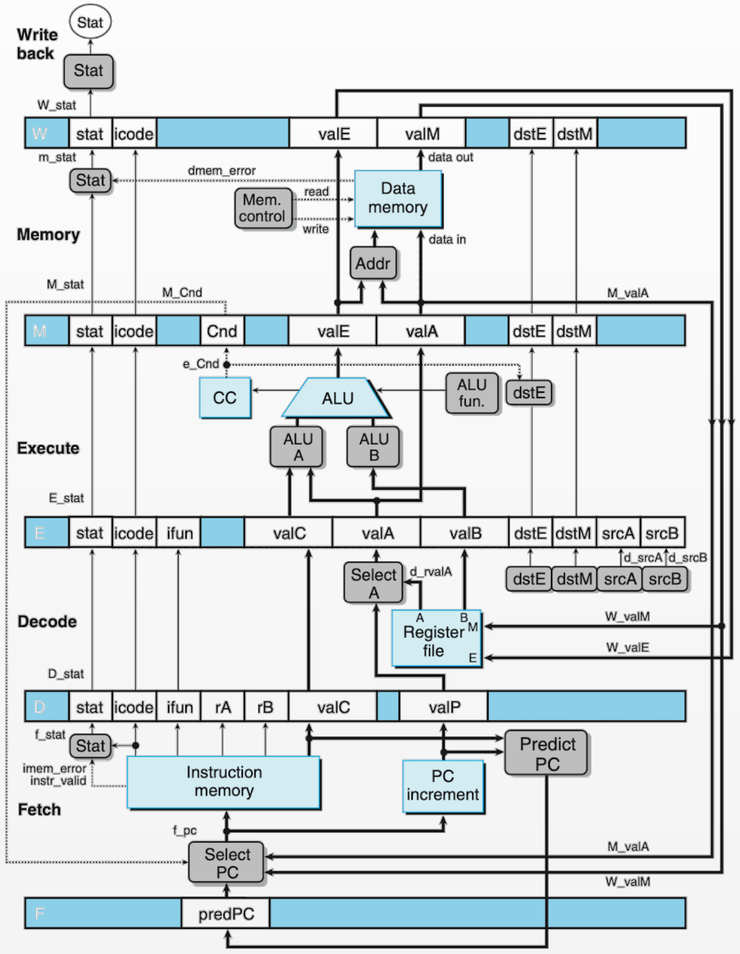
1. 插入流水线寄存器

- SEQ+插入流水线寄存器得到PIPE-

7. 实现流水线

1. 插入流水线寄存器





7. 实现流水线

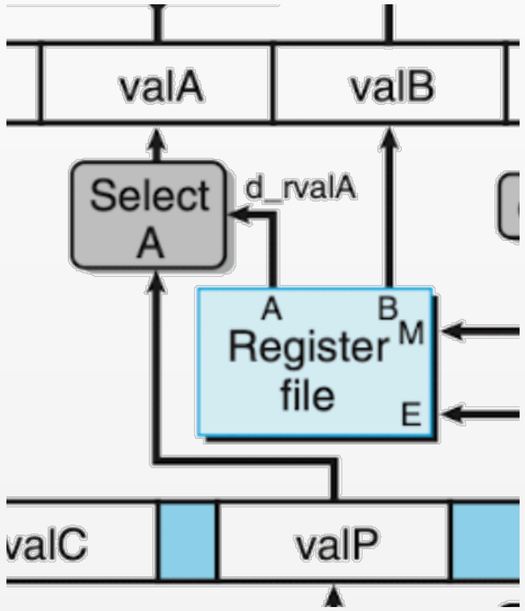
2. 命名信号

- 小写前缀代表当前阶段刚计算出的值
- 大写前缀代表当前阶段对应流水线寄存器存储的值
- 小写信号产生在大写之后

7. 实现流水线

3. 简化信号

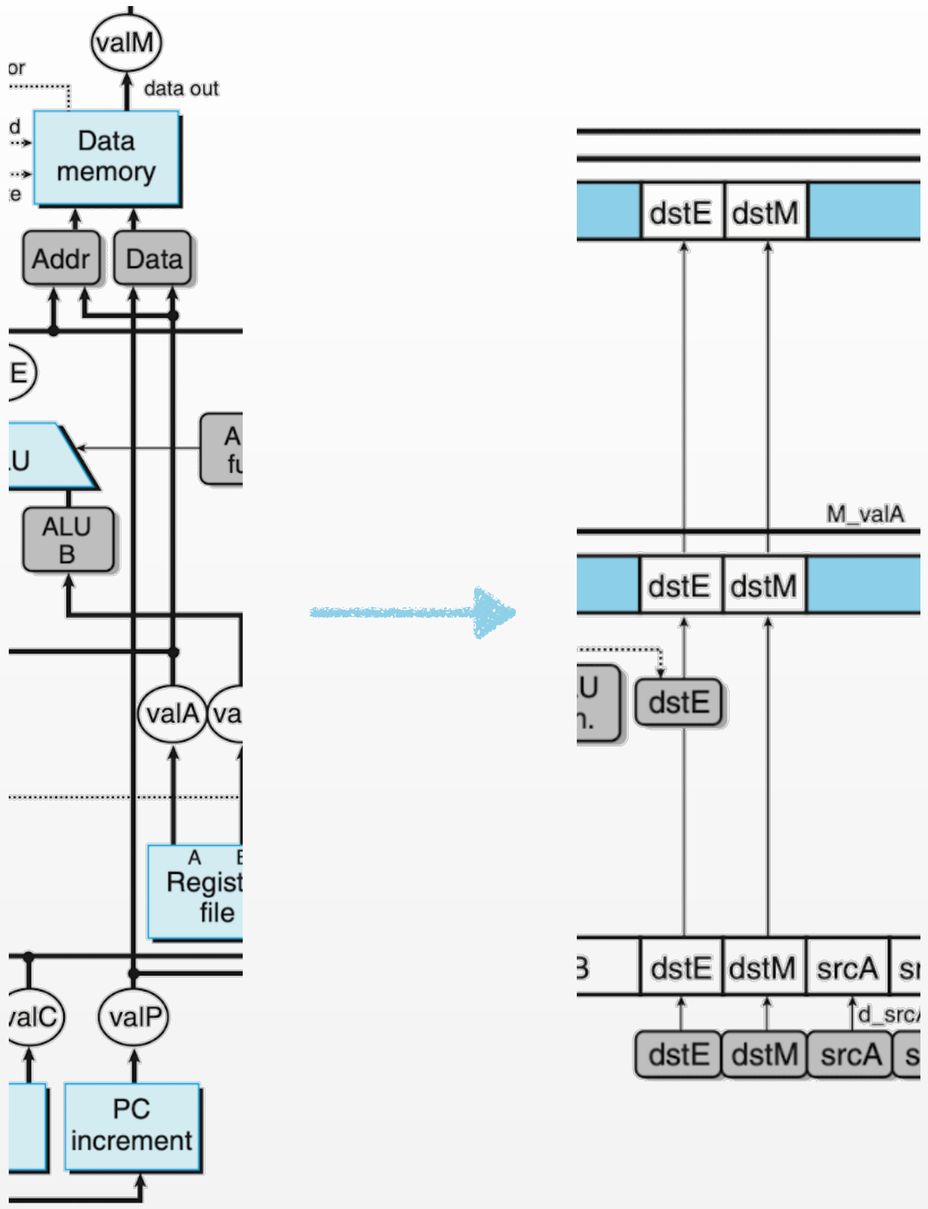
- valP被合并进valA
- CALL, JXX用到，这两个指令都不用读取寄存器



7. 实现流水线

3. 简化信号

- 后续需要保留的信号直接连到下一个流水线寄存器
- 后续不需要的信号不用在下一个流水线寄存器中保存
- dstE, dstM 在Decode阶段产生，并被保留到Write Back



7. 实现流水线 - 流水线阶段

Fetch

- Select current PC
  - predPC
  - M_valA (branch_mispred detected in M)
  - W_valM (ret in W)
- Read instruction
- Compute incremented PC

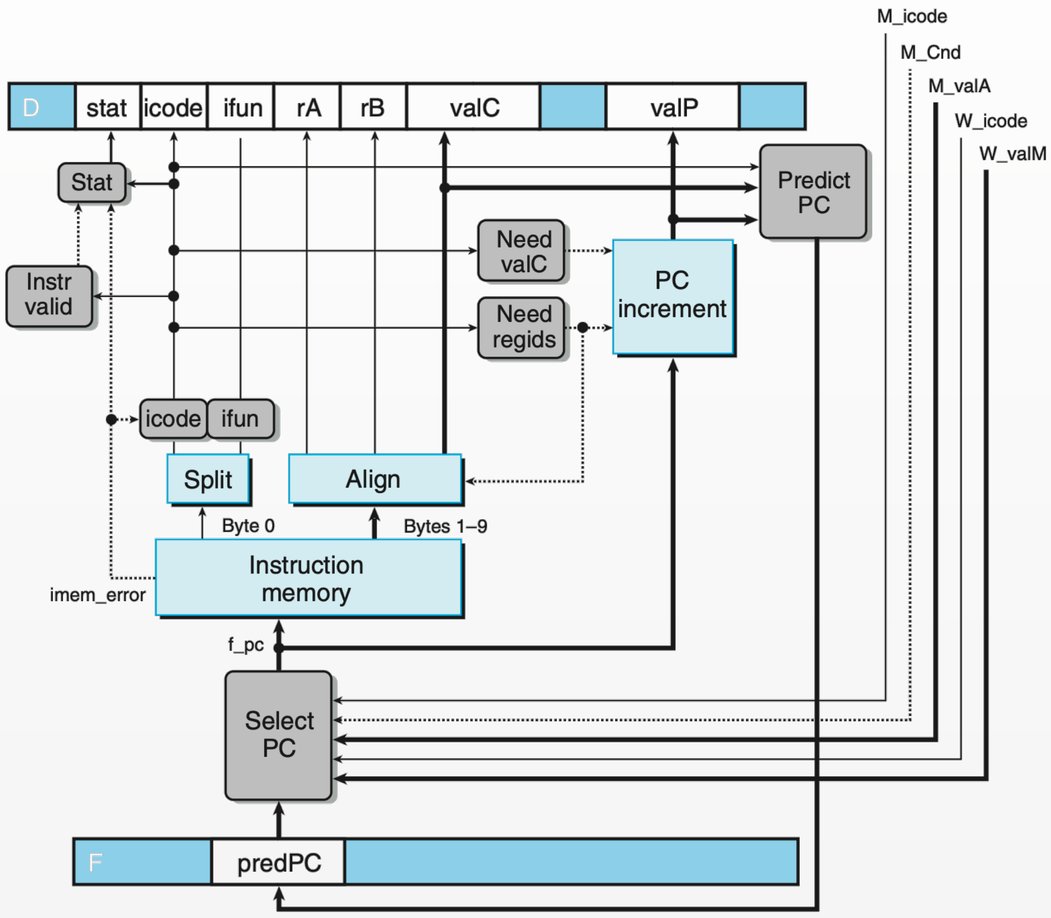


Figure 4.57 **PIPE PC selection and fetch logic.** Within the one cycle time limit, the processor can only predict the address of the next instruction.

7. 实现流水线 - 流水线阶段

Decode

- Read program registers
- Forwarding logic
- Select between valA and valP

Data word	Register ID	Source description
e_valE	e_dstE	ALU output
m_valM	M_dstM	Memory output
M_valE	M_dstE	Pending write to port E in memory stage
W_valM	W_dstM	Pending write to port M in write-back stage
W_valE	W_dstE	Pending write to port E in write-back stage

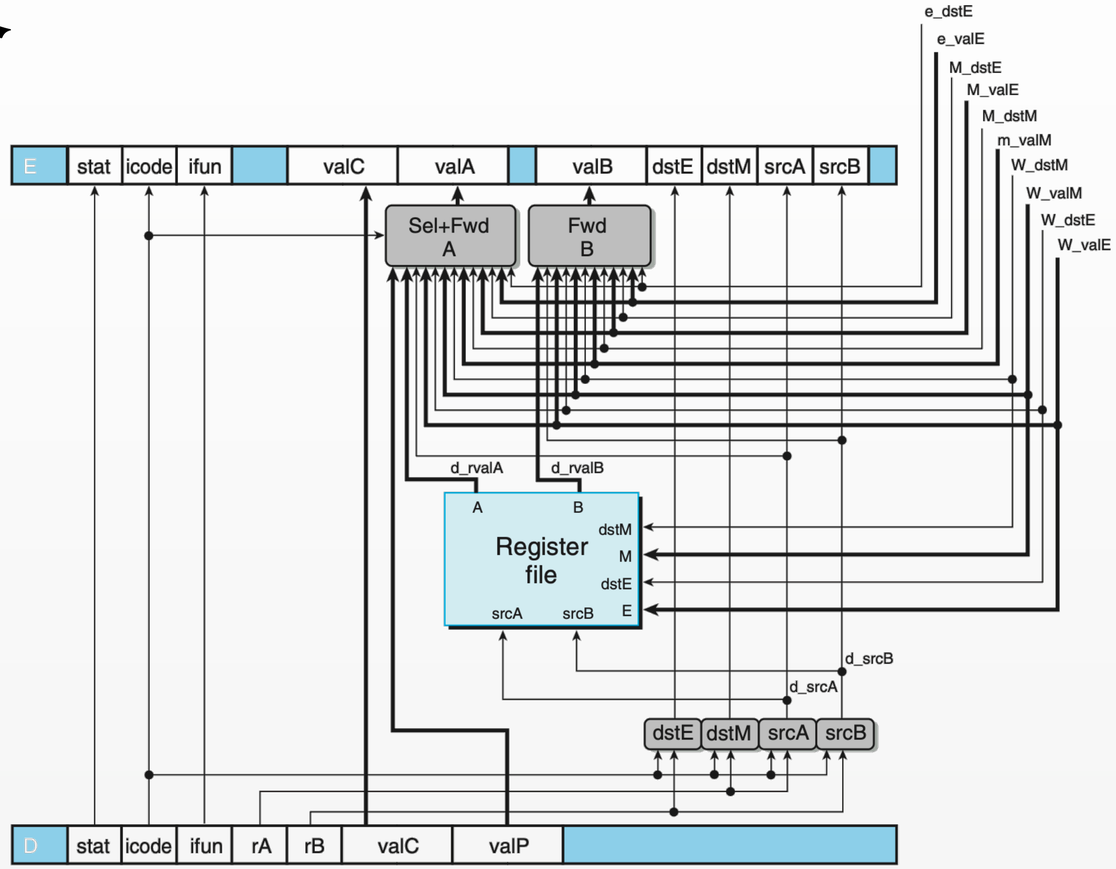


Figure 4.58 **PIPE decode and write-back stage logic.** No instruction requires both valP and the value read from register port A, and so these two can be merged to form the signal valA for later stages. The block labeled "Sel+Fwd A" performs this task and also implements the forwarding logic for source operand valA. The block labeled "Fwd B" implements the forwarding logic for source operand valB. The register write locations are specified by the dstE and dstM signals from the write-back stage rather than from the decode stage, since it is writing the results of the instruction currently in the write-back stage.

7. 实现流水线 - 流水线阶段

Execute

- Operate ALU
- Set CC
  - m_stat and W_stat for exception handling

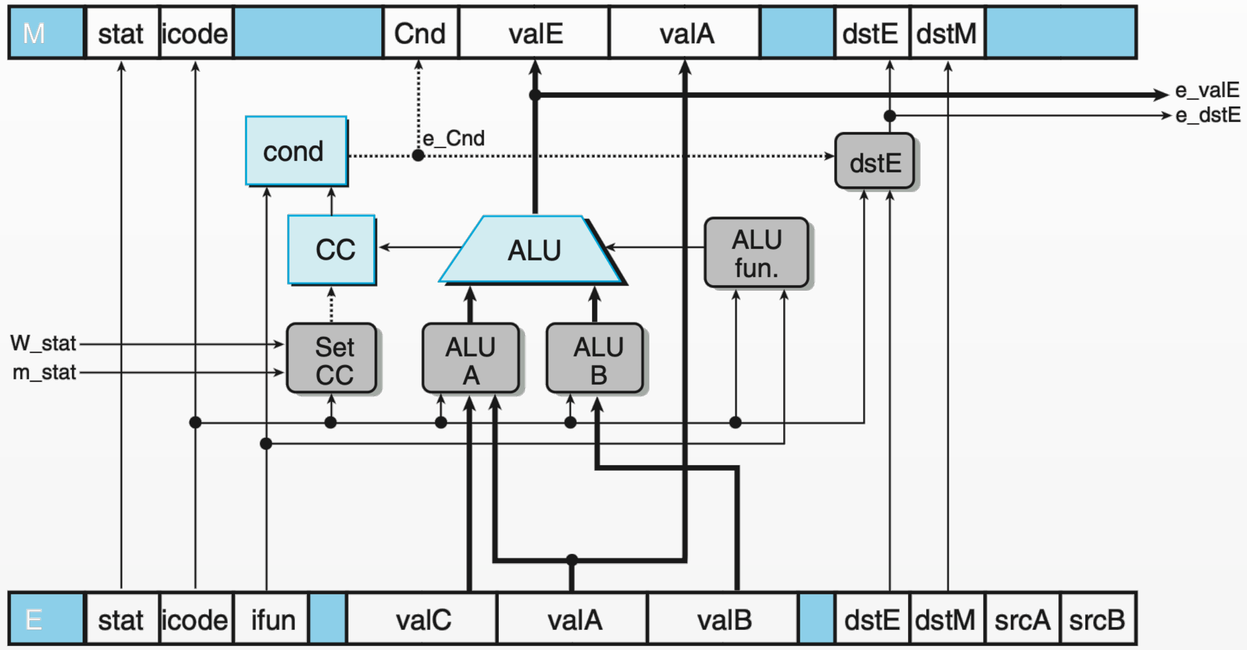


Figure 4.60 **PIPE execute stage logic.** This part of the design is very similar to the logic in the SEQ implementation.

7. 实现流水线 - 流水线阶段

Memory

- Read or write data memory

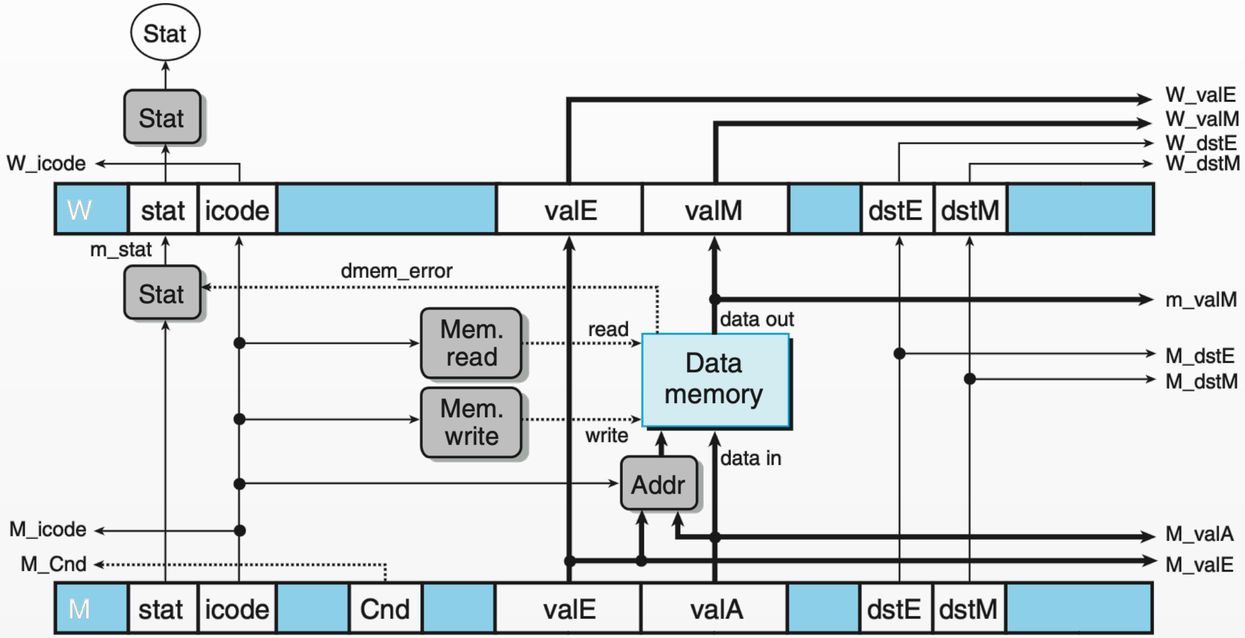
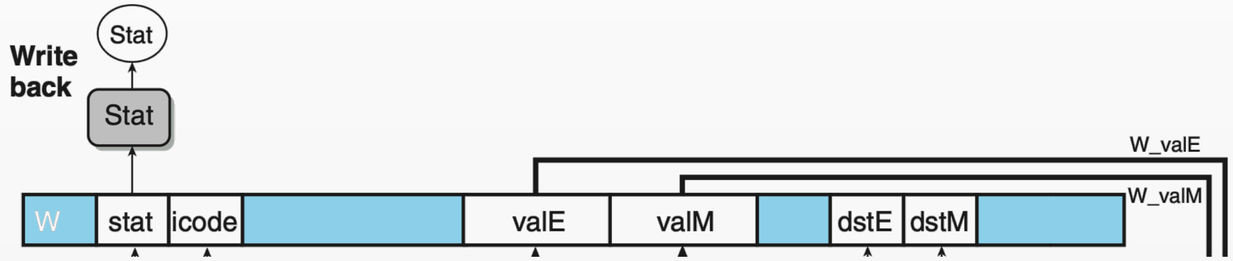


Figure 4.61 **PIPE memory stage logic.** Many of the signals from pipeline registers M and W are passed down to earlier stages to provide write-back results, instruction addresses, and forwarded results.

7. 实现流水线 - 流水线阶段

Write Back

- Update register file



8. Feedbacks

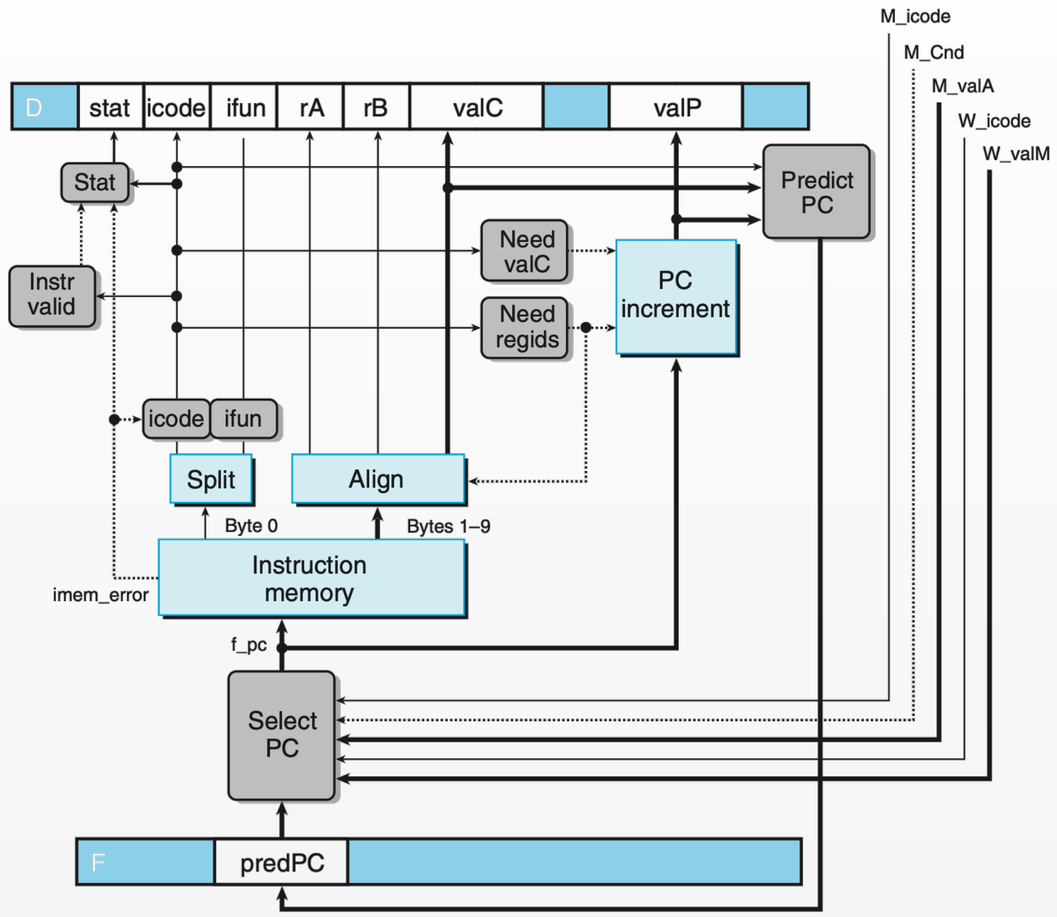


Figure 4.57 PIPE PC selection and fetch logic. Within the one cycle time limit, the processor can only predict the address of the next instruction.

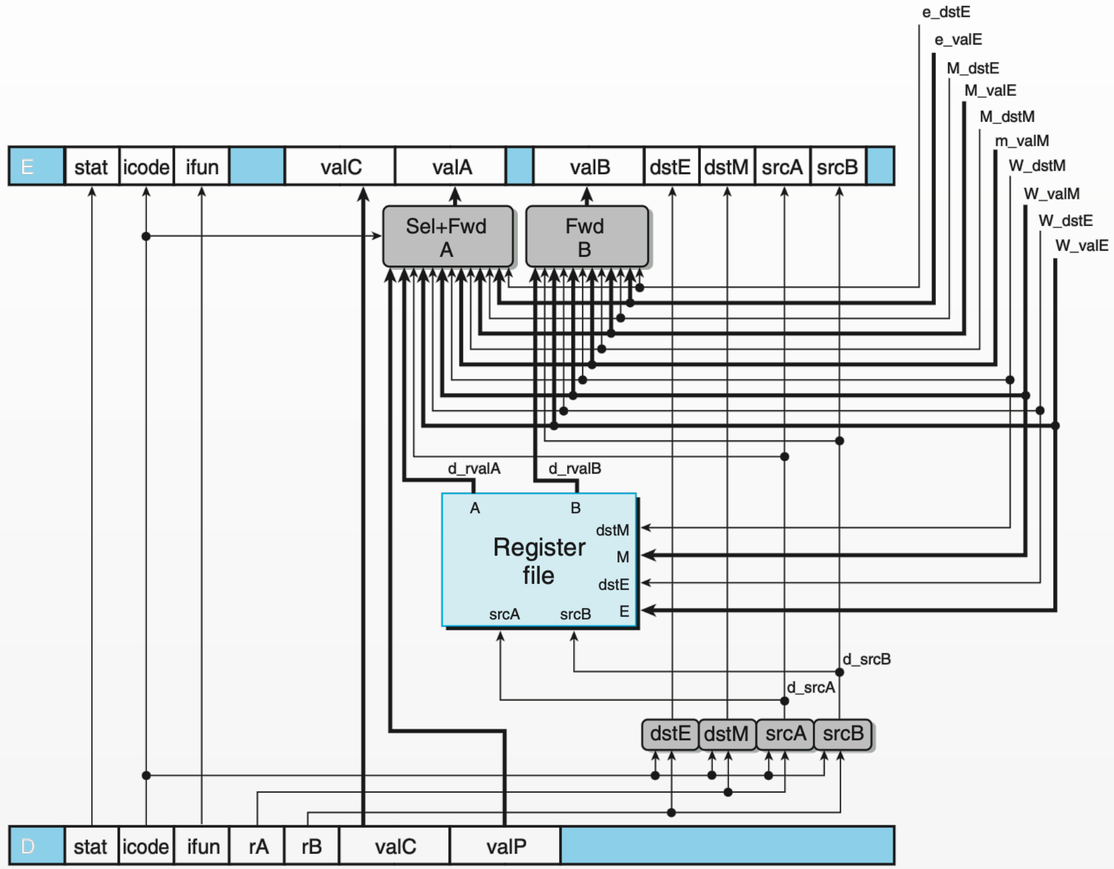


Figure 4.58 PIPE decode and write-back stage logic. No instruction requires both valP and the value read from register port A, and so these two can be merged to form the signal valA for later stages. The block labeled "Sel+Fwd A" performs this task and also implements the forwarding logic for source operand valA. The block labeled "Fwd B" implements the forwarding logic for source operand valB. The register write locations are specified by the dstE and dstM signals from the write-back stage rather than from the decode stage, since it is writing the results of the instruction currently in the write-back stage.

9. PC prediction

f_predPC

always taken logic

```
word f_predPC = [
  # Always taken
  f_icode in { IJXX, ICALL } : f_valC;
  # Default
  1 : f_valP;
];
```

f_pc

select from **M_valA**（always taken预测错误，这个结果最早在M register中），**W_valM**（ret），**F__predPC**(fallback)

```
word f_pc = [
  # Mispredicted branch. Fetch at incremented PC
  M_icode == IJXX && !M_Cnd : M_valA;
  # Completion of RET instruction
  W_icode == IRET : W_valM;
  # Default: Use predicted value of PC
  1 : F_predPC;
];
```

谢谢!